
The problem of reinforcement learning with many discrete actions

Diogo S. Carvalho

INESC-ID

Instituto Superior Técnico, University of Lisbon
Portugal

`diogo.s.carvalho@tecnico.ulisboa.pt`

Pedro P. Santos

INESC-ID

Instituto Superior Técnico, University of Lisbon
Portugal

`pedro.pinto.santos@tecnico.ulisboa.pt`

Pedro A. Santos

INESC-ID

Instituto Superior Técnico, University of Lisbon
Portugal

`pedro.santos@tecnico.ulisboa.pt`

Francisco S. Melo

INESC-ID

Instituto Superior Técnico, University of Lisbon
Portugal

`fmelo@inesc-id.pt`

Abstract

Reinforcement learning performs well in small discrete action spaces and continuous action spaces. However, reinforcement learning methods do not deal well with large discrete and unstructured action spaces. There is a fundamental difficulty with learning over large discrete action spaces, which revolve around a structuring and generalization problem. Then, computational difficulties of reinforcement learning appear, which revolve around an optimization problem. In this work, we go through the problems of reinforcement learning over many discrete actions, systematize the current solutions and conclude with research directions for the future.

Keywords: reinforcement learning with function approximation

Acknowledgements

This work was supported by national funds through FCT, Fundação para a Ciência e a Tecnologia, under projects UIDB/50021/2020 and PTDC/CCI-COM/7203/2020 and scholarship 2020.05360.BD.

1 Introduction

Many real world problems have large discrete actions spaces. Examples are recommendation systems, industrial plants, natural language problems, centralized multi-agent control and combinatorial optimization [Dulac-Arnold et al., 2015].

Reinforcement learning performs well in small discrete action spaces and continuous action spaces. However, there is a fundamental difficulty with learning over large discrete action spaces, which we call the generalization problem. A possible solution to the generalization problem is to structure the actions by learning features from which to generalize.

Suppose we could learn the action structure, in the form of features, and be able to generalize from it. Then, computational difficulties of reinforcement learning appear, which we call the optimization problem. Both problems have been identified in the literature [Dulac-Arnold et al., 2015]. In this work, we go deeper in the discussion of the problem of reinforcement learning over many discrete actions and review the solutions that have been proposed, mostly orthogonally, since the identification of the problems. We contribute by organizing, comparing and discussing the various solutions deeper and under a novel taxonomy. We conclude with research directions for the future.

We note that we preclude combinatorial optimization problems and solutions from our work as it is a special, even harder, case with state-dependent action spaces and their solutions have been surveyed before [Mazyavkina et al., 2021].

2 Background

A Markov decision process is a sequential decision making process formalized as a tuple $(\mathcal{X}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{X}$ is the state space, $\mathcal{A}$ is the action space, $P : \mathcal{X} \times \mathcal{A} \rightarrow \Delta(\mathcal{X})^1$ is the transition probability distribution, $r : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$ is the expected reward and $\gamma \in (0, 1)$ is a discount factor. $\mathcal{X}$ and $\mathcal{A}$ can be discrete or continuous.

A policy maps states to actions in a possibly stochastic manner. The value of a policy is measured by its value function $Q_\pi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}$. The value of executing action a at state x and then following the policy π is the expected total sum of discounted rewards obtained and verifies the fixed point equation $Q_\pi(x, a) = \mathbb{E}_\pi[r(x, a) + \gamma Q_\pi(x', a') - Q_\pi(x, a) \mid x = x, a = a]$, where the next state x' is distributed according to $P(x, a)$ and the next action a' is distributed according to π .

There is an optimal policy π^* with value $Q^* := Q_{\pi^*}$ verifying the fixed point equation $Q^*(x, a) = \mathbb{E}[r(x, a) + \gamma \max_{a' \in \mathcal{A}} Q^*(x', a') - Q^*(x, a) \mid x = x, a = a]$. A policy that is greedy with respect to Q^* is optimal. Finding the optimal policy is called the Markov decision problem. Reinforcement learning methods aim at solving the Markov decision problem and can be categorized under two classes. Value based methods build on the fact that the optimal value function directly provides an optimal policy and consider a family of value functions Q parameterized by $w \in \mathbb{R}^K$. Policy based methods approximate a policy π_w within a family of policies Π . Formally, rather than a family of policies, Π is a family of distribution generating functions $\pi_w : \mathcal{X} \rightarrow \Delta(\mathcal{A})$ from which to sample actions.

3 Problem

The size of a Markov decision problem is a determinant factor of the difficulty of the problem. The aforementioned dependency is known as the curse of dimensionality. Many state and action pairs to visit put pressure on the exploration versus exploitation tradeoff and raise sample and computational efficiency issues.

3.1 Generalization

To deal with the dimensionality, function approximation architectures are a transversal element in reinforcement learning. Deep neural networks, for example, are designed to learn internal representation features of the input with respect to the output. Reinforcement learning with function approximation is able to handle diverse state observations by producing good feature representations of states. These feature representations of states are used upstream to generalize information across states and downstream to compute action value or probabilities.

Successful reinforcement learning applications have either (i) discrete and small action spaces or (ii) continuous action spaces. In fact, function approximation architectures in RL do not attempt to structure and generalize the action space because in (i), generalizing information across actions is not prement; in (ii), the action space already has a structure.

On top of the problem of action generalization, we make the case for another problem of reinforcement learning with many discrete actions. The problem is computational and would exist even if the generalization problem was solved.

¹ $\Delta(S)$ denotes the set of probability distributions over the set S . If S is finite, $\Delta(S)$ may be referred to as the simplex on S .

3.2 Optimization

Value based methods require the maximization of value over actions at the next state. Additionally, for selecting the greedy action with respect to the value, the same optimization must be made. If the action space is finite, the optimization can be done in linear time with respect to the action space. Still, a sub-linear time complexity would be valuable. If the action space is continuous, the optimization burden is usually avoided and policy based methods are preferred.

In discrete action spaces, policy based methods output, for a certain state, a probability distribution over actions. Consequently, they also require an operation of linear complexity. Additionally, to select an action according to the computed probabilities also requires linear time. If the action space is continuous, policy based methods usually approximate a policy that directly outputs an action given a state, leveraging the already present structure over the action space.

4 Solutions

Throughout the years, several solutions have been constructed to contour or mitigate one or both problems identified. We start with approaches to learn action features from which to generalize.

4.1 Learning action features

As stated before, architectures such as deep neural networks are designed to learn good representation features over the input. Reinforcement learning methods use such capabilities to learn a map $\phi : \mathcal{X} \rightarrow \mathbb{R}^K$, for some $K \in \mathbb{N}$, that structures the state space in a sense that depends on the method being used and generalize from it.

In value based methods, the features ϕ are learned to structure states according to their action values. The same features ϕ are then used to output action values as $Q(x, a) = \phi(x)^T w_a$, where $w_a \in \mathbb{R}^K$ are also learned. There is, however, a way to generalize actions in value based methods. We can, instead, learn features $\psi : \mathcal{X} \times \mathcal{A} \rightarrow \mathbb{R}^K$ such that $Q(x, a) = \psi(x, a)^T w$. The features ψ thus learn to generalize information across state action pairs with respect to their value.

He et al. [2015] proposed an approach for learning action features in value based methods which falls under the generalization framework just mentioned. The authors propose to learn independent state features ϕ and action features $\lambda : \mathcal{A} \rightarrow \mathbb{R}^K$ and then combine them as $Q(x, a) = \phi(x)^T \lambda(a)$. Narasimhan et al. [2015] proposed to consider learning action features that are dependent of the state features through a map $\xi : \mathbb{R}^K \times \mathcal{A} \rightarrow \mathbb{R}^K$ such that $Q(x, a) = \xi(\phi(x), a)^T w$.

Policy based methods also leverage on function approximation capabilities to learn features ϕ that structures states with respect to action probabilities. The features are then used to compute a probability distribution over actions as $\pi(x) = \sigma(\phi(x)^T W)$, where W is a matrix in $\mathbb{R}^K \times \mathbb{R}^{|\mathcal{A}|}$ and $\sigma : \mathbb{R}^{|\mathcal{A}|} \rightarrow \Delta(\mathcal{A})$ is a non-linear distribution generating function such as the softmax. Policy based action generalization is harder than value based, since actions are not an input of the policy.

Lagoudakis and Parr [2003] proposed a policy based approach able to generalize actions. If we assume the possibility to perform multiple rollouts from a state. We can compute a rollout for each action and compute all the action values for each state x in $\mathcal{Y} \subset \mathcal{X}$ and consequently its optimal action a_x^* . We obtain a labelled dataset $\mathcal{D} = \{((x, a), \mathbf{1}_{a=a_x^*}), x \in \mathcal{Y}, a \in \mathcal{A}\}$ to perform supervised learning on. We obtain state action features ψ and get the policy $\pi(x) = \sigma([\psi(x, a)^T z]_{a \in \mathcal{A}})$.

The methods presented herein do not address the optimization problem identified before. The value based solutions still require to compute every action value and the policy based solution also requires the computation of every action score. We noticed that reinforcement learning methods are well suited for small discrete action spaces. The methods from the next section try to deal with large discrete action spaces by considering smaller instances thereof.

4.2 Reducing the action space

Several methods have argued that the factorization of a discrete action space allows for the computation of action values or probabilities according to each factor of the representation [Pazis and Parr, 2011, Pierrot et al., 2020, Sharma et al., 2017]. Assume the action space is composed of N discrete actions and suppose a factorization of $\mathcal{A}$ as $\mathcal{A}'^N$, with smaller factors $\mathcal{A}'$. Particularly, assume a binary decomposition. Then an action value or probability can be computed for each A_i in parallel. Such methods assumed that an handcrafted binarization was provided beforehand.

Departing from the idea that reinforcement learning methods are better able to generalize the state space rather than the action space, methods have shifted the burden of action space size to state space size by sequentializing the decision making [Majeed and Hutter, 2020, Bamford and Ovalle, 2021]. Consider again a decomposition of the action space $\mathcal{A}$ as $\mathcal{A}'^N$. Rather than, from state x , directly outputting an action or action value for each action a in $\mathcal{A}$, we can consider outputting an action $a[1]$ from x , then for $a[2]$ from the extended state $(x, a[1])$, until the last action factor is outputted. The reward given for each action selection is zero until the complete sequence $a = (a[1], \dots, a[N])$ is computed.

Other works try to reduce the action space by eliminating redundant or irrelevant actions for each state throughout learning. Zahavy et al. [2018] proposes a multi-armed contextual bandit problem for action elimination from an action elimination signal that is part of the problem. The method does not try to learn an action elimination signal itself. Vinyals et al. [2019] proposed to prune the action space available for each state using expert data. Both methods, once actions are eliminated, do not allow for retrieval. Affordances for reinforcement learning were proposed by Khetarpal et al. [2020] and aim at pruning the action space for each decision [Khetarpal et al., 2020]. However, the method also requires intentions of the agent, previously known, or expert data. Finally, Van de Wiele et al. [2020] proposes a way to select a subset of actions to consider before computing the action values in value based methods.

Finally, Czarnecki et al. [2018] and Farquhar et al. [2020] proposed to consider starting learning with a small action space $\bar{\mathcal{A}}$, a subset of the original $\mathcal{A}$, and then increasing it throughout learning until the original action space is reached. The method aims at allowing curriculum learning and easier generalization since it allows action values to be transferred along the action spaces. However, none of the works specify how to effectively learn a way to grow the action space.

Alternatively to consider smaller instances of the action space, methods on the next section consider a continuous embedded representation of the actions, where policy based methods perform well.

4.3 Embedding the action space

We move on to the methods that aim at considering a continuous action space representation of the discrete action space, in order to use policy based methods that directly output an action without evaluating every action value or probability.

Dulac-Arnold et al. [2015] made the point that if actions were embedded in a continuous space, we could use a policy based method for continuous action spaces to directly compute actions from states. Suppose every action $a \in \mathcal{A}$ has an embedding $\hat{a}$ in $\mathbb{R}^K$. From the policy $\pi : \mathcal{X} \rightarrow \mathbb{R}^K$ we can compute an $\hat{a}_x = \pi(x) \in \mathbb{R}^K$ and select the set $\hat{\mathcal{A}}_x \subset \mathcal{A}$ of k -nearest discrete neighbours of $\hat{a}_x$ with logarithmic time complexity in the size of $\mathcal{A}$. We then evaluate the value of the actions in $\hat{\mathcal{A}}_x$ and select the maximal one. The work does not propose a way to construct the embedding space itself.

Tennenholtz and Mannor [2019] proposed a way to construct the embedding space, assumed to exist in the previous work. The idea is that actions can be embedded according to their context in expert demonstrations, through a process akin to word embedding in the context of natural language processing. The action context considered is with respect to neighbouring actions in the expert. The method thus assumes that expert demonstrations of the task are available.

Finally, Chandak et al. [2019] proposed a way to learn the embedding space throughout the learning process. The method learns functions $g : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R}^K$ and $f : \mathbb{R}^K \rightarrow \Delta(\mathcal{A})$ such that $f \circ g$ learns, in a supervised manner, to classify labelled $((x_t, x_{t+1}), a_t)$, following that a supervised signal is easier to learn than a reward signal. Then, using any policy based method, we learn $\pi : \mathcal{X} \rightarrow \mathbb{R}^K$ such that $f \circ \pi$ outputs a discrete action in $\mathcal{A}$.

Methods in subsection 4.1 are designed to directly learn a structure over the action space, given by the feature representation of actions, from which to generalize. The feature representation itself could be used as the embedding structure on top of which to use the sub-linear time complexity action selection process of Dulac-Arnold et al. [2015].

By considering continuous representations of the action space, the present section eases the use of policy based methods with large discrete action spaces and is a promising research direction. However, continuous representations of the action space are not usually available, neither are sufficient expert demonstrations for a given reinforcement learning task. Chandak et al. [2019] proposed a method that learns the continuous structure itself. Learning is supervised with respect to state transitions. Therefore, actions are structured according to an implicitly learned model of the dynamics. It would be interesting to consider ways of embedding the actions through a model free supervised signal, such as the action context suggested by Tennenholtz and Mannor [2019], without the need of expert demonstrations.

5 Conclusion

In this work, we started by arguing that, when the action space is large and discrete, reinforcement learning must deal with two different problems. The problems had been identified in the literature by Dulac-Arnold et al. [2015]. We went deeper in such discussion. Then, we systematized the solutions that have been made available for reinforcement learning with large discrete action spaces. We made the point that, besides from the traditional generalization power given by function approximation methods of learning, some methods have tried to make the action space smaller and others have tried to make it continuous, inspired by the success stories of reinforcement learning in both cases.

A direct but motivating research direction is to combine methods from Section 4.1, that learn features that structure the action space, and thus address the generalization problem, with the methods from Section 4.3 that build on an embedded continuous structure of actions and allow the use of policy based methods, addressing the optimization problem. Then, still regarding structure and generalization of actions, it would be interesting to see non-euclidean structures capturing

more meaningful proximity between actions, for instance Poincaré embeddings [Nickel and Kiela, 2017] or an extension of state bisimulations to actions Ferns et al. [2004]. Regarding methods from Section 4.2, that aim at reducing the action space, it would be interesting to alleviate their assumptions and allow for intrinsic learning of action elimination signals of the environment, for instance in a process conversely related to curiosity [Pathak et al., 2017].

It would also be worth understanding which of the different methods reviewed can be combined; when such combination is helpful; if their use is algorithm and learning architecture agnostic. Finally, the suggestion of benchmark problems for reinforcement learning with large discrete action spaces could potentially drive research further. Then, an experimental study accross the benchamrk problems of the standard reinforcement learning methods and the ones discussed in the document could clarify the performance and efficiency benefits of each.

References

- Christopher Bamford and Alvaro Ovalle. Generalising discrete action spaces with conditional action trees. In *2021 IEEE Conference on Games (CoG)*, pages 1–8. IEEE, 2021.
- Yash Chandak, Georgios Theodorou, James Kostas, Scott Jordan, and Philip Thomas. Learning action representations for reinforcement learning. In *International conference on machine learning*, pages 941–950. PMLR, 2019.
- Wojciech Czarnecki, Siddhant Jayakumar, Max Jaderberg, Leonard Hasenclever, Yee Whye Teh, Nicolas Heess, Simon Osindero, and Razvan Pascanu. Mix & match agent curricula for reinforcement learning. In *International Conference on Machine Learning*, pages 1087–1095. PMLR, 2018.
- Gabriel Dulac-Arnold, Richard Evans, Peter Sunehag, and Ben Coppin. Reinforcement learning in large discrete action spaces. 2015.
- Gregory Farquhar, Laura Gustafson, Zeming Lin, Shimon Whiteson, Nicolas Usunier, and Gabriel Synnaeve. Growing action spaces. In *International Conference on Machine Learning*, pages 3040–3051. PMLR, 2020.
- Norm Ferns, Prakash Panangaden, and Doina Precup. Metrics for finite markov decision processes. In *UAI*, volume 4, pages 162–169, 2004.
- Ji He, Jianshu Chen, Xiaodong He, Jianfeng Gao, Lihong Li, Li Deng, and Mari Ostendorf. Deep reinforcement learning with a natural language action space. *arXiv preprint arXiv:1511.04636*, 2015.
- Khimya Khetarpal, Zafarali Ahmed, Gheorghe Comanici, David Abel, and Doina Precup. What can i do here? a theory of affordances in reinforcement learning. In *International Conference on Machine Learning*, pages 5243–5253. PMLR, 2020.
- Michail G Lagoudakis and Ronald Parr. Reinforcement learning as classification: Leveraging modern classifiers. In *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*, pages 424–431, 2003.
- Sultan Javed Majeed and Marcus Hutter. Exact reduction of huge action spaces in general reinforcement learning. *arXiv preprint arXiv:2012.10200*, 2020.
- Nina Mazyavkina, Sergey Sviridov, Sergei Ivanov, and Evgeny Burnaev. Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*, 134:105400, 2021.
- Karthik Narasimhan, Tejas Kulkarni, and Regina Barzilay. Language understanding for text-based games using deep reinforcement learning. *arXiv preprint arXiv:1506.08941*, 2015.
- Maximillian Nickel and Douwe Kiela. Poincaré embeddings for learning hierarchical representations. *Advances in neural information processing systems*, 30, 2017.
- Deepak Pathak, Pulkit Agrawal, Alexei A Efros, and Trevor Darrell. Curiosity-driven exploration by self-supervised prediction. In *International conference on machine learning*, pages 2778–2787. PMLR, 2017.
- Jason Pazis and Ronald Parr. Generalized value functions for large action sets. In *ICML*, 2011.
- Thomas Pierrot, Valentin Macé, Jean-Baptiste Sevestre, Louis Monier, Alexandre Laterre, Nicolas Perrin, Karim Beguir, and Olivier Sigaud. Factored action spaces in deep reinforcement learning. 2020.
- Sahil Sharma, Aravind Suresh, Rahul Ramesh, and Balaraman Ravindran. Learning to factor policies and action-value functions: Factored action space representations for deep reinforcement learning. *arXiv preprint arXiv:1705.07269*, 2017.
- Guy Tennenholtz and Shie Mannor. The natural language of actions. In *International Conference on Machine Learning*, pages 6196–6205. PMLR, 2019.
- Tom Van de Wiele, David Warde-Farley, Andriy Mnih, and Volodymyr Mnih. Q-learning in enormous action spaces via amortized approximate maximization. *arXiv preprint arXiv:2001.08116*, 2020.
- Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *Nature*, 575(7782):350–354, 2019.
- Tom Zahavy, Matan Haroush, Nadav Merlis, Daniel J Mankowitz, and Shie Mannor. Learn what not to learn: Action elimination with deep reinforcement learning. *Advances in Neural Information Processing Systems*, 31, 2018.